

Edge AI Refrigerator Telemetry System

A TinyML-Based Embedded Monitoring and
Anomaly Detection System

Project Documentation

Muhammad Haseeb

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

2026

Contents

1	Introduction	6
1.1	Abstract	6
1.2	Background	6
1.3	Project Motivation	6
1.4	Problem Statement	7
1.5	Project Objectives	7
1.6	Scope of the Project	8
1.7	Key Technologies	8
2	System Architecture and Design	9
2.1	System Overview	9
2.2	System Architecture	9
2.3	Functional Architecture	9
2.3.1	Sensing Layer	9
2.3.2	Edge Processing Layer	9
2.3.3	Machine Learning Layer	10
2.3.4	Communication Layer	10
2.3.5	Application Layer	10
2.4	Hardware Components	10
2.4.1	ESP32	10
2.4.2	DHT22 Temperature and Humidity Sensor	10
2.4.3	LDR Illumination Sensor	10
2.4.4	Supporting Components	10
2.5	Hardware Configuration	11
2.6	Software Architecture	11
2.6.1	Embedded Firmware	11
2.6.2	Machine Learning Development Environment	12
2.6.3	Communication and JSON Layer	12
2.7	Data Flow	12
2.8	Edge AI Design Principle	12
2.9	Prototype and Simulation Environment	13
3	Data Acquisition and Dataset Preparation	13
3.1	Overview	13

3.2	Sensor Data Acquisition	14
3.3	Data Collection Scenarios	14
3.4	Multivariate Sensor Data	15
3.5	Data Labeling	15
3.6	Dataset Organization	15
3.7	Data Preprocessing	16
3.8	Time-Window Based Processing	16
3.9	Feature Extraction	16
3.10	Edge Impulse Development Workflow	17
3.11	Impulse Configuration	17
3.12	Dataset Splitting	18
3.13	Importance of Dataset Diversity	18
3.14	Data Preparation for Embedded Deployment	18
3.15	Dataset Quality Considerations	19
3.16	Data-to-Model Pipeline	19
3.17	Summary	19
4	TinyML Model Development and Deployment	19
4.1	Overview	19
4.2	Input Features	19
4.3	Edge Impulse Configuration	20
4.4	Feature Extraction	20
4.5	Model Architecture	20
4.6	Training Process	21
4.7	Training Metrics	21
4.7.1	Accuracy	21
4.7.2	Precision	21
4.7.3	Recall	22
4.7.4	F1-Score	22
4.8	Confusion Matrix	22
4.9	Classification Output	22
4.10	Model Optimization for ESP32	22
4.11	Model Deployment	23
4.12	Embedded Inference Process	23
4.13	Advantages of TinyML Deployment	24

4.14 Summary	24
5 Embedded Implementation and Real-Time Monitoring	24
5.1 Overview	24
5.2 ESP32 Firmware Structure	24
5.3 System Initialization	24
5.4 Sensor Acquisition	25
5.5 Sensor Validation	25
5.6 Data Preparation for Inference	25
5.7 TinyML Inference on ESP32	25
5.8 Classification Handling	25
5.9 Real-Time Telemetry	26
5.10 JSON Telemetry Format	26
5.11 Wi-Fi Communication	26
5.12 Dashboard Architecture	26
5.13 Dashboard Data Representation	27
5.14 Real-Time Monitoring Loop	27
5.15 Monitoring Sequence	28
5.16 Local Inference and Remote Visualization	28
5.17 Error Handling and Connectivity	28
5.18 Wokwi Simulation	29
5.19 Complete Runtime Architecture	29
5.20 System Operation Example	29
5.21 Implementation Summary	29
6 Testing, Results, and Evaluation	30
6.1 Overview	30
6.2 Testing Methodology	30
6.3 Sensor Testing	30
6.4 ESP32 Firmware Testing	30
6.5 Machine Learning Model Evaluation	31
6.6 Confusion Matrix Evaluation	32
6.7 Model Confidence	32
6.8 Normal Condition Testing	32
6.9 Abnormal Condition Testing	32
6.10 Door-Open Pattern Observation	33

6.11	Embedded Model Testing	33
6.12	Wi-Fi Communication Testing	33
6.13	JSON Telemetry Testing	34
6.14	Web Dashboard Testing	34
6.15	End-to-End System Testing	34
6.16	End-to-End Test Cases	34
6.17	Model Resource Evaluation	34
6.18	Observed Results	35
6.19	Performance Summary	35
6.20	Limitations	35
6.20.1	Limited Sensor Set	36
6.20.2	Dataset Dependency	36
6.20.3	Prototype Environment	36
6.20.4	Anomaly Interpretation	36
6.21	Discussion	36
6.22	Conclusion of Evaluation	37
7	Challenges and Solutions	37
7.1	Sensor Data Variability	37
7.2	Dataset Collection and Labeling	37
7.3	Difference Between Normal and Abnormal Conditions	37
7.4	Embedded Resource Constraints	37
7.5	Consistency Between Training and Deployment	38
7.6	Real-Time Inference Integration	38
7.7	Wi-Fi Connectivity	38
7.8	Dashboard Synchronization	38
7.9	Simulation Versus Physical Hardware	38
7.10	Summary of Challenges	38
8	Future Improvements	38
8.1	Physical Refrigerator Deployment	39
8.2	Additional Sensors	39
8.3	Multiclass Fault Detection	39
8.4	Larger and More Diverse Dataset	39
8.5	Historical Data Visualization	40
8.6	Alert and Notification System	40

8.7	Energy Consumption Monitoring	40
8.8	Secure IoT Communication	40
8.9	Over-the-Air Firmware Updates	40
8.10	Low-Power Operation	40
8.11	Predictive Maintenance	40
9	Conclusion	41
A	Hardware Connections	42
B	Example JSON Telemetry	42
C	Firmware Structure	42
D	Edge Impulse Model Report	43
E	Final Project Summary	43

1 Introduction

1.1 Abstract

This project is a small refrigerator monitor that thinks for itself. An ESP32 microcontroller reads temperature and humidity from a DHT22 sensor and light levels from an LDR, then runs a lightweight machine learning model directly on the board to decide whether the refrigerator is behaving normally. Because the sensors respond to different things, the model can pick up combined patterns, such as light rising while the temperature drifts upward, which is what you would expect when a door is left open. A single fixed threshold would struggle to describe that.

The model was built in Edge Impulse and deployed to the ESP32, so inference happens on the device itself and does not depend on a cloud service. The ESP32 then sends its readings and its verdict over Wi-Fi as JSON, and a web dashboard shows the current state.

Taken together, the project walks through a complete Edge AI workflow: collecting sensor data, preparing a dataset, extracting features, training and evaluating a model, deploying it to a microcontroller, running real-time inference, and presenting the results to a user. It is a prototype, but one that could reasonably grow into predictive maintenance, food-storage monitoring, smarter alerting, or energy-aware refrigeration.

1.2 Background

A refrigerator has one main job, which is to keep its contents at a steady, safe temperature. When something changes inside it, whether the temperature, the humidity or the amount of light, that change says something about what the appliance is doing. A door left open, for instance, lets in light and warm air, which wastes energy and puts stored food at risk.

Most monitoring systems deal with this using fixed thresholds: if the temperature goes above a set value, raise an alert. That is easy to build, but it only looks at one number at a time. It cannot tell a harmless blip from a sustained problem, and it ignores how the different signals relate to each other. Temperature, humidity and illumination together carry far more information than any one of them alone.

This project looks at that gap through the lens of Edge AI and TinyML. The ESP32 serves as both the processing unit and the communication device, the sensors supply the environmental data, and a trained model replaces much of the hand-written rule logic. The model is developed in Edge Impulse using labeled sensor data and then deployed to the ESP32, which classifies conditions as they happen. Readings and results are sent over Wi-Fi in JSON, and a dashboard displays them.

The project draws on several areas of computer systems engineering at once: embedded programming, sensor interfacing, wireless communication, machine learning and web-based monitoring. The goal is not only a working refrigerator monitor, but a demonstration of the full path from physical sensing to on-device inference to user-facing visualization.

1.3 Project Motivation

The starting point was the growing interest in running machine learning on small, resource-constrained devices. Many IoT systems simply stream raw sensor data to a server and let it do the heavy lifting. That works, but it makes the device dependent on a network connection and on a remote machine for every decision.

Edge AI takes the opposite view and moves some of the intelligence into the device. Here the ESP32 runs the model itself, so it can form a basic judgment about the refrigerator without a server looking at every reading.

A refrigerator turned out to be a good test case. Its temperature, humidity and light levels are continuous signals that inexpensive sensors can capture, and they are well suited to training a model small enough for a microcontroller.

A second motivation was to cover the whole lifecycle of a TinyML application rather than stopping at a model trained on a laptop. The project goes through:

1. Collecting data from physical sensors.
2. Preparing and organizing the dataset.
3. Extracting useful features from the sensor signals.
4. Training a machine learning model.
5. Evaluating the trained model.
6. Deploying the model to an ESP32.
7. Running inference directly on the device.
8. Sending telemetry and inference results over Wi-Fi.
9. Visualizing the results on a web dashboard.

1.4 Problem Statement

Conventional refrigerator monitoring leans on fixed thresholds or raw sensor readings, and these often miss patterns that span several signals. A rise in temperature together with a sudden jump in light may mean the door has been opened. Slow, unusual shifts in the environment over a period of time may point to some other abnormal condition. Picking these out with individual threshold values is difficult.

The problem this project addresses is therefore to design and build a low-cost embedded system that collects refrigerator telemetry and uses a lightweight machine learning model, running on the device itself, to flag abnormal patterns. The ESP32 is the central platform and Edge Impulse is used to develop and deploy the model.

1.5 Project Objectives

The project set out to:

1. Design a sensor-based system for monitoring a refrigerator's environmental conditions.
2. Interface temperature, humidity and illumination sensors with an ESP32.
3. Collect and organize sensor data representing normal and abnormal conditions.
4. Develop a machine learning model with the Edge Impulse platform.
5. Apply suitable preprocessing and feature extraction to the sensor signals.
6. Deploy the trained TinyML model to the ESP32.
7. Run inference locally on the embedded device.

8. Use Wi-Fi to transmit telemetry and inference results.
9. Represent sensor and model outputs in a structured JSON format.
10. Build a web dashboard that shows the telemetry and the detected condition.
11. Demonstrate a complete Edge AI pipeline, from physical sensing through local inference to remote visualization.

1.6 Scope of the Project

The project covers a prototype refrigerator monitor built around an ESP32. It concentrates on environmental telemetry and on using machine learning to recognize abnormal conditions.

On the hardware side, the system includes the ESP32, a DHT22 temperature and humidity sensor, and an LDR-based light sensing circuit. On the software side, it includes embedded C++ firmware, sensor data processing, Wi-Fi communication, JSON telemetry, model development in Edge Impulse, and a web monitoring interface.

The machine learning component recognizes patterns in the available sensor data. It does not attempt detailed fault diagnosis. A detected anomaly should be read as a sign that someone may want to take a look at the refrigerator. The project is a prototype and is not meant to replace commercial refrigerator controls or safety systems. Its purpose is to show how TinyML and Edge AI ideas work in a practical embedded IoT setting.

1.7 Key Technologies

Table 1 lists the main technologies and what each was used for.

Table 1: Technologies used in the project	
Technology	Purpose
ESP32	Main processing platform and Wi-Fi communication.
DHT22	Measures temperature and relative humidity.
LDR	Detects changes in illumination linked to door activity.
Edge Impulse	Dataset management, feature extraction, model training and TinyML deployment.
TinyML	Allows machine learning inference on the resource-constrained ESP32.
C++	Firmware development for the ESP32.
Wokwi	Simulation and early testing of the ESP32-based system.
Wi-Fi	Wireless link between the ESP32 and the monitoring interface.
JSON	Structured format for sensor telemetry and inference results.
Web dashboard	Displays live sensor readings and the detected condition.

2 System Architecture and Design

2.1 System Overview

The system is a chain of connected stages: sensing, processing, inference, communication and visualization. The ESP32 sits in the middle of it. It takes measurements from the DHT22 and the LDR, prepares them for the model, and runs the trained TinyML model on the device to classify what it sees. Alongside that, the ESP32 uses its built-in Wi-Fi to send selected telemetry and the inference result as JSON, which the web dashboard picks up for display.

In order, the stages are sensor data acquisition, preprocessing, feature extraction, TinyML inference, local classification, Wi-Fi communication, JSON telemetry generation and web-based visualization. The high-level flow is:

Sensors → ESP32 → Preprocessing → TinyML Model → Inference → JSON/Wi-Fi → Web Dashboard

2.2 System Architecture

The guiding idea is to keep the intelligence as close to the data as possible. Rather than sending every raw reading to another machine for classification, the ESP32 does the classification itself.

The DHT22 supplies temperature and humidity, and the LDR supplies light information. Together these are the system's view of the refrigerator environment. The ESP32 reads them at regular intervals and arranges them into a suitable format. Depending on the sampling setup, several readings can be grouped into a time window before feature processing. That window goes to the deployed Edge Impulse model, which returns a classification of the pattern it observed. The ESP32 then combines the raw telemetry with the model output into one JSON object and transmits it over Wi-Fi for the dashboard.

DHT22 + LDR → ESP32 Data Acquisition → Preprocessing → Edge Impulse TinyML Model
→ Local Inference → JSON / Wi-Fi → Web Dashboard

Figure 1: High-level architecture of the Edge AI Refrigerator Telemetry System

2.3 Functional Architecture

It helps to think of the system as five layers.

2.3.1 Sensing Layer

This layer is the DHT22 and the LDR. The DHT22 measures temperature and relative humidity. The LDR provides a light-related measurement that reflects changes in lighting inside the refrigerator.

2.3.2 Edge Processing Layer

The ESP32 is the edge processing layer. It reads the sensors, organizes the data, preprocesses it and runs the model. It is worth stressing that it is more than a data logger here: the intelligence-related work happens on this device.

2.3.3 Machine Learning Layer

This layer holds the model developed in Edge Impulse. Sensor data is turned into features and passed to the trained model, which recognizes patterns similar to those in the training dataset. The embedded application uses its output to decide the current classification.

2.3.4 Communication Layer

Communication is handled by the ESP32's built-in Wi-Fi. Once the readings and the inference result are available, the firmware packages them as a JSON object, which is easy to transmit, parse and display in external applications.

2.3.5 Application Layer

The web dashboard is the application layer and gives the user a readable view of the system. It typically shows the current temperature, humidity and illumination level, the detected condition and model classification, and recent telemetry.

2.4 Hardware Components

2.4.1 ESP32

The ESP32 was chosen because it pairs real processing capability with integrated Wi-Fi, which suits both Edge AI and IoT work. In this project it reads sensor measurements, processes and prepares the data, executes the TinyML model, determines the classification, generates the telemetry and communicates with the monitoring system over Wi-Fi.

Using one device for sensing, inference and communication keeps the architecture simple, and it shows that lightweight machine learning workloads can run on a microcontroller.

2.4.2 DHT22 Temperature and Humidity Sensor

The DHT22 measures the temperature and relative humidity of the monitored environment. Temperature matters most, since abnormal changes can point to a change in the refrigerator's state. Humidity adds more environmental context and contributes to the overall sensor pattern the model sees. The sensor talks to the ESP32 digitally, and the firmware requests a new measurement periodically.

2.4.3 LDR Illumination Sensor

A light dependent resistor (LDR) tracks illumination. It is useful here because the amount of light inside a refrigerator changes noticeably when the door is opened. The LDR is not treated as a standalone door sensor. Its readings are one part of the overall telemetry signal, combined with temperature and humidity to identify patterns associated with abnormal conditions.

2.4.4 Supporting Components

Depending on whether the build is physical or simulated, a few supporting items are needed: a breadboard or prototyping environment, jumper wires, a suitable resistor for the LDR circuit,

a USB connection for programming the ESP32, and a computer for model development and firmware deployment.

2.5 Hardware Configuration

The wiring is straightforward. The DHT22 connects to a digital GPIO pin on the ESP32, and the LDR circuit connects to an ADC-capable input so its changing voltage can be sampled. GPIO assignments must stay consistent across the circuit, the firmware and the simulation. In the final circuit the DHT22 data line is on GPIO 15 and the analog output of the LDR module is on GPIO 34.

Table 2: Sensor interface overview

Component	Interface	Purpose
DHT22	Digital GPIO	Temperature and humidity measurement
LDR	Analog input	Illumination measurement
ESP32 Wi-Fi	Wireless	Telemetry transmission and dashboard communication

2.6 Software Architecture

The software side has four parts: embedded firmware, machine learning development, communication, and the dashboard.

2.6.1 Embedded Firmware

The firmware controls the whole embedded workflow. Its main steps are:

1. Initialize the ESP32 peripherals.
2. Initialize the DHT22 and LDR interfaces.
3. Connect to Wi-Fi.
4. Read sensor values at the configured sampling interval.
5. Prepare the data for inference.
6. Run the deployed machine learning model.
7. Obtain the classification result.
8. Build the telemetry JSON object.
9. Transmit or expose the data to the monitoring application.
10. Repeat continuously.

The firmware is the coordinator between the physical sensors, the TinyML model and the dashboard.

2.6.2 Machine Learning Development Environment

Edge Impulse handles the machine learning side. It manages sensor datasets, defines processing blocks, extracts features, trains the model, evaluates it and generates an embedded deployment package. The general workflow is:

Sensor Data → Dataset → Processing → Feature Extraction → Training → Evaluation →
ESP32 Deployment

2.6.3 Communication and JSON Layer

The communication component links the embedded device to the monitoring interface. Telemetry is represented as a JSON object with fields such as these:

```
{
  "temp": <value>,
  "light": <value>,
  "status": "<classification>",
  "confidence": <value>
}
```

These are the field names used by the firmware in this project. JSON is lightweight and human-readable, and web applications can consume it easily.

2.7 Data Flow

Data flow begins when the sensors observe the refrigerator environment. The DHT22 measures temperature and humidity, the LDR measures illumination, and the ESP32 gathers these and ties them to the current sampling window. The readings are processed into the format the deployed model expects, and the model runs locally on the ESP32 to produce a classification. The firmware then combines the sensor measurements with that result, serializes it as JSON and sends it over Wi-Fi. The dashboard receives the telemetry and presents it in a form a user can read.

$$\begin{aligned} &\text{Environment} \rightarrow \text{Sensors} \rightarrow \text{ESP32} \rightarrow \text{Preprocessing} \\ &\rightarrow \text{TinyML Inference} \rightarrow \text{JSON} \rightarrow \text{Wi-Fi} \rightarrow \text{Dashboard} \end{aligned} \quad (1)$$

2.8 Edge AI Design Principle

The key design decision was to run inference on the ESP32 rather than depending on a remote server. In a typical cloud setup, sensor data would be uploaded continuously and the model would run remotely. Here the trained model is embedded in the microcontroller and executes locally. That brings several practical advantages:

- Inference does not depend on continuous cloud connectivity.
- Sensor data is processed locally.
- Less data has to be sent for machine learning decisions.
- Classification results are available sooner on the device.
- It shows machine learning running on a resource-constrained platform.

Wi-Fi is still used, but for telemetry visualization and remote monitoring. The core classification stays on the edge device.

2.9 Prototype and Simulation Environment

The system was developed and tested in the Wokwi simulator. Simulation is a convenient way to check firmware, sensor interfaces, data handling and system logic without constantly rewiring physical hardware, and it is especially handy early in development. The architecture itself is designed around a real ESP32 and real sensor interfaces, so moving from simulation to a physical build is a realistic next step.

The first simulation used three potentiometers on ADC pins 34, 35 and 32 to stand in for temperature, humidity and illumination. The raw readings were scaled in code to 20–50 °C, 20–90 % and 0–5000, and a simple threshold check printed an alert on the serial monitor (Figure 2). That was enough to test scaling and serial output before the real DHT22 was added.

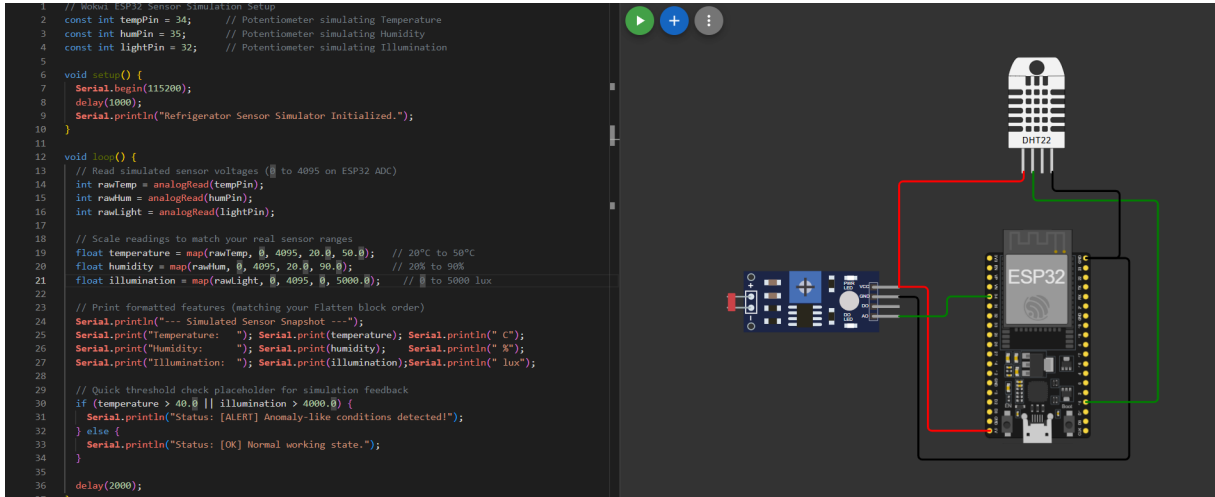


Figure 2: First Wokwi prototype: potentiometers simulate the three sensors, and a simple threshold check prints a status message.

3 Data Acquisition and Dataset Preparation

3.1 Overview

An Edge AI system is only as good as the data it is trained on. The dataset for this project was built from the same kinds of sensor signals the system sees in operation: temperature, humidity and illumination. These three complement one another. Temperature describes the thermal condition, humidity adds environmental context, and illumination captures changes associated with the door.

The aim throughout was to keep the training environment close to the final embedded system. The workflow was:

Sensor Data Collection → Data Cleaning → Labeling → Data Segmentation → Feature
Extraction → Model Training

3.2 Sensor Data Acquisition

The ESP32 served as the data acquisition platform. During collection, it periodically read the connected sensors and recorded the temperature, relative humidity and illumination level at each sampling point. A single observation can be written as

$$X_i = [T_i, H_i, L_i] \quad (2)$$

where T_i , H_i and L_i are the temperature, humidity and illumination measured at sample i .

A single reading says little about how things are changing, so consecutive observations are considered together. A sequence of n observations is

$$X = \begin{bmatrix} T_1 & H_1 & L_1 \\ T_2 & H_2 & L_2 \\ \vdots & \vdots & \vdots \\ T_n & H_n & L_n \end{bmatrix} \quad (3)$$

where n is the number of samples in the chosen observation window.

In the final Wokwi circuit the DHT22 is connected to GPIO 15 and the LDR module to the analog pin GPIO 34 (Figure 3). A data-collection sketch reads the sensors once per second and prints one CSV row per sample over the serial port, with the header `timestamp,temperature, humidity,illumination`. If the DHT22 returns an invalid reading, the sketch substitutes a default cold-fridge value (4.0 °C and 45 % humidity) so that no invalid rows reach the dataset. Temperature and humidity are changed with the DHT22 sliders in Wokwi, and illumination is the raw 12-bit ADC value from the LDR module (0–4095). Edge Impulse labels the illumination axis as lux, but the values are ADC counts. The recorded CSV files were uploaded to Edge Impulse with the CSV Wizard, so no physical device was connected to the project during data collection.

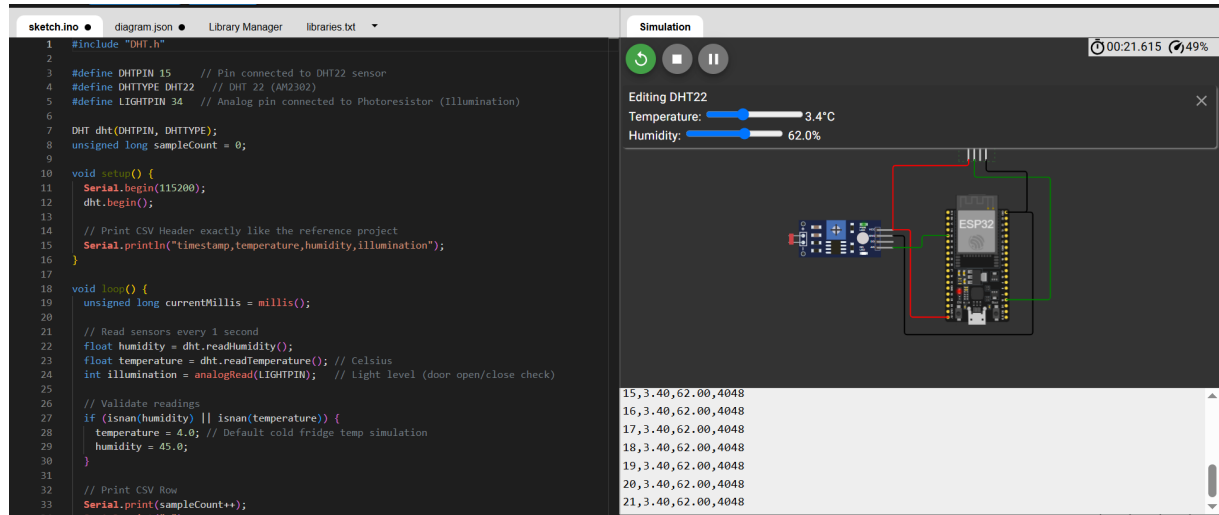


Figure 3: Final Wokwi circuit and data-collection sketch. The DHT22 sliders set temperature and humidity, and the serial monitor shows one CSV row per second.

3.3 Data Collection Scenarios

The dataset was organized around two broad conditions:

- **Normal condition:** sensor readings that represent the expected operating environment.
- **Abnormal condition:** sensor patterns that represent unusual environmental behavior.

An abnormal condition does not necessarily mean a specific hardware fault. It simply means a pattern that differs from the normal behavior in the training data.

A door being opened is a good example. The LDR sees a clear change in light, and the temperature may drift depending on how long the door stays open. The combination says more than either signal alone.

3.4 Multivariate Sensor Data

Using several signals together is one of the central ideas of the project. With temperature alone, the system could notice a change but would know little about its cause. With illumination alone, it could suspect the door state has changed but would not see the environmental effect. Combining them gives a broader picture:

$$P = f(T, H, L) \quad (4)$$

where P is the observed sensor pattern and f describes the relationship between the signals. This is where machine learning earns its place in an embedded application, since the model can learn those relationships instead of relying on hand-picked thresholds.

3.5 Data Labeling

Each sample or sample window was labeled with the condition it represented. Labels must stay consistent across dataset preparation, training and embedded inference. The binary setup uses two classes:

- **NORMAL:** behavior associated with the expected refrigerator condition.
- **ANOMALY:** a pattern that differs from the normal condition.

These labels give the supervised learning algorithm examples of the conditions it has to tell apart.

3.6 Dataset Organization

The collected data is stored as structured records of sensor measurements, each linked to its class. Table 3 shows the general shape of a record. Each observation window is associated with its class label during preparation.

Table 3: Example sensor dataset structure

Sample	Temperature	Humidity	Light
1	T_1	H_1	L_1
2	T_2	H_2	L_2
3	T_3	H_3	L_3
$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	T_n	H_n	L_n

In total, 18 min 11 s of data was collected, as one recording per class for training and one per class for testing. The training set holds `normal_data_train` (label NORMAL, 7 min 28 s) and `abnormal_data_train` (label ANOMALY, 7 min 4 s). The test set holds `normal_data_test` (1 min 53 s) and `abnormal_data_test` (1 min 46 s). Data is sampled at 1 Hz. Figure 4 shows the Edge Impulse data acquisition page, with the raw signals of the anomaly recording: the illumination channel repeatedly jumps between low and high values.

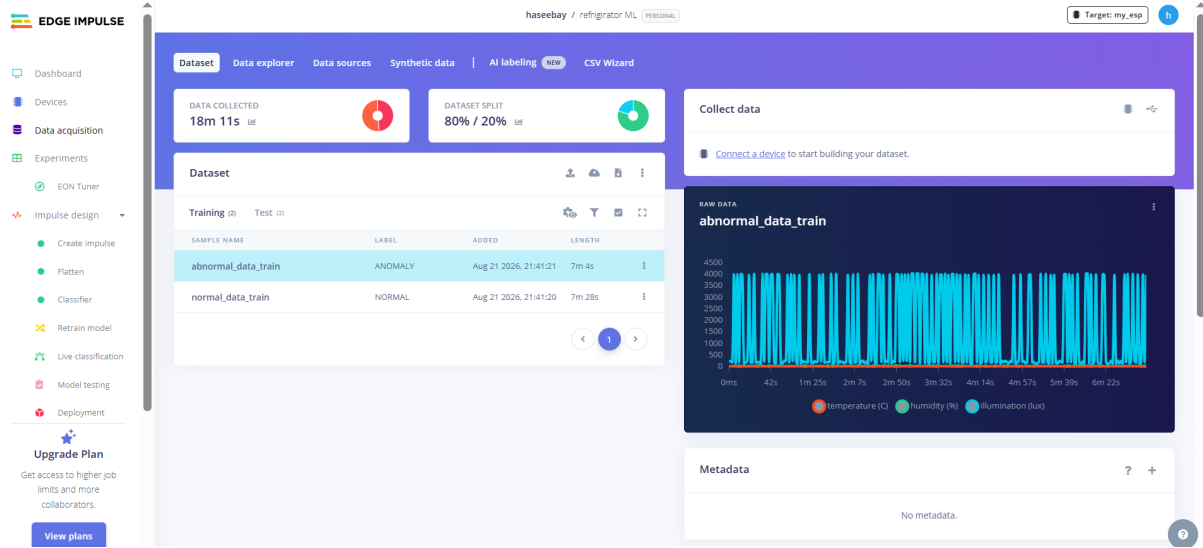


Figure 4: Edge Impulse data acquisition page: two training recordings (NORMAL and ANOMALY), an 80%/20% split and the raw signal plot.

3.7 Data Preprocessing

Raw sensor data picks up noise and small fluctuations, so it is processed according to the Edge Impulse pipeline before reaching the model. That includes organizing readings into a consistent format, grouping consecutive measurements into time windows, keeping the sensor channels aligned, removing invalid or unusable observations, and converting measurements into the numerical form the model expects. Getting this right matters, because any mismatch between training data and deployment data can reduce the reliability of embedded inference.

3.8 Time-Window Based Processing

Rather than treat each reading as an independent observation, the system can handle a run of consecutive samples as a single input window. This is useful because refrigerator conditions change over time. Opening the door may change illumination immediately, while temperature responds more slowly, and a window lets the model see both together. For a window of n samples,

$$W = \{X_1, X_2, \dots, X_n\} \quad (5)$$

where each X_i holds the sensor measurements at one sampling point. The model can then learn characteristics of the whole window and not just a single instant.

3.9 Feature Extraction

Feature extraction turns the sensor window into a representation that makes the differences between classes easier to learn. Depending on the Edge Impulse processing block chosen, this can

mean statistical or signal-related characteristics of the window. In general form,

$$F = g(W) \quad (6)$$

where W is the sensor window, g is the processing and feature-extraction operation, and F is the resulting feature representation. This stage matters even more for TinyML, because the final model has to fit within the ESP32's computational and memory limits.

In this project the processing block is Flatten, which summarizes each axis of a window with a set of statistics (by default the average, minimum, maximum, RMS, standard deviation, skewness and kurtosis). Seven statistics for each of the three axes give 21 features per window, which is exactly the size of the classifier's input layer. Figure 5 shows the feature explorer: windows of the NORMAL and ANOMALY classes form two largely separate clusters, with a small area of overlap at the boundary.

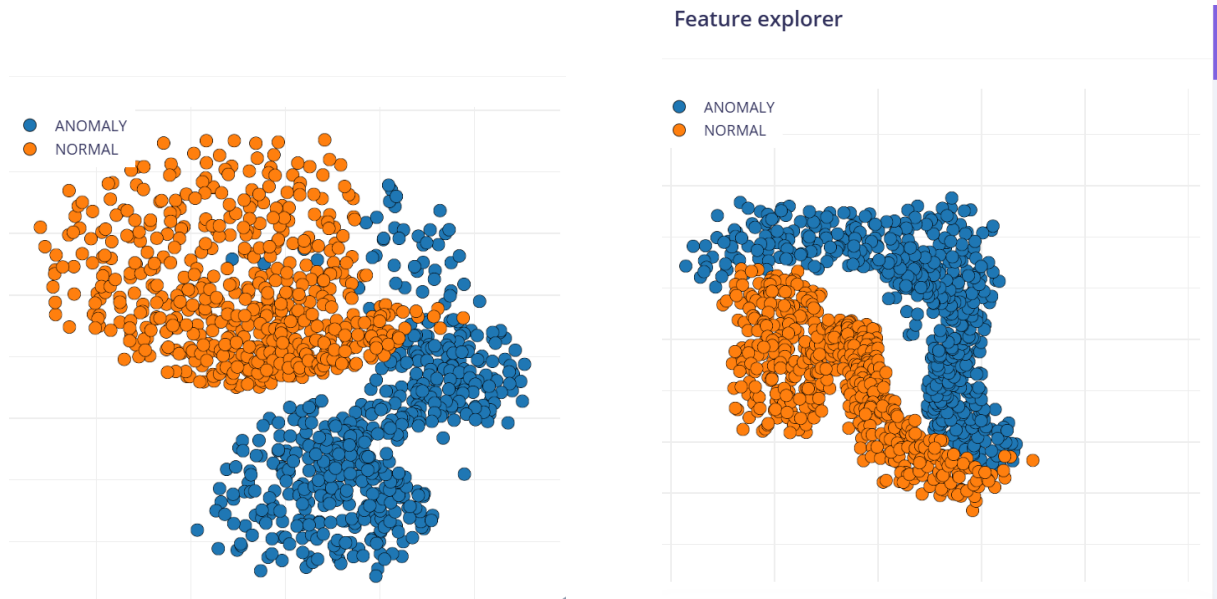


Figure 5: Feature explorer views of the NORMAL and ANOMALY windows.

3.10 Edge Impulse Development Workflow

Edge Impulse was the main development environment for the TinyML side. The workflow ran through data acquisition, dataset organization, impulse configuration, signal preprocessing, feature extraction, model training, model evaluation and deployment. It makes it possible to go from raw sensor data to an embedded model without hand-implementing every stage.

3.11 Impulse Configuration

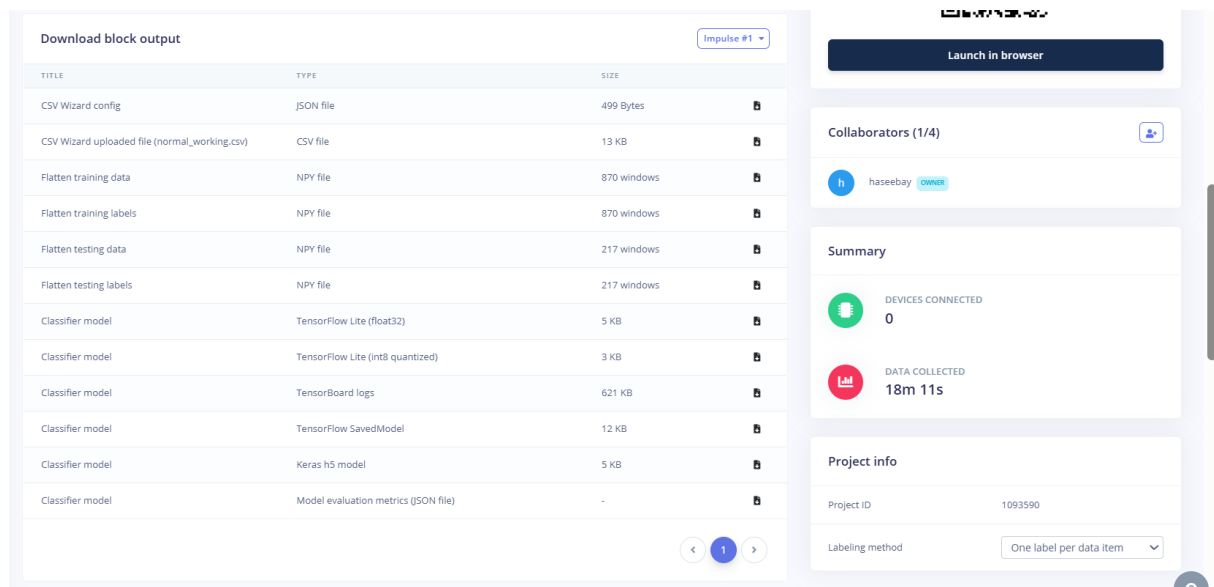
An impulse defines how raw sensor data becomes the input to the learning algorithm. It consists of an input configuration, a processing block and a learning block. The input here is the refrigerator telemetry. The processing block turns it into features, and the learning block trains the classifier on those features.

The impulse takes three input axes (temperature, humidity and illumination) sampled at 1 Hz. Each window is 2 s long, which is two samples per axis, and the window moves forward by 1 s. This produces 870 training windows and 217 test windows (Figure 6). The processing block is

Flatten and the learning block is a neural network classifier; the full parameters are in Table 4. These must not be changed between training and the deployed firmware unless the model is retrained. The firmware follows the same layout: it gives the classifier a raw feature signal of six values, which is two samples of three axes.

3.12 Dataset Splitting

The data is divided into training, validation and testing portions. Training data is what the algorithm learns from. Validation data is used during development to see how the model does on samples it has not directly learned from. A separate test set gives a final indication of performance on unseen samples. Edge Impulse split the recordings 80%/20% between training and testing, which gave 870 training windows and 217 test windows (Figure 6). Validation figures during training are computed by Edge Impulse on a held-out part of the training windows.



TITLE	TYPE	SIZE
CSV Wizard config	JSON file	499 Bytes
CSV Wizard uploaded file (normal_working.csv)	CSV file	13 KB
Flatten training data	NPY file	870 windows
Flatten training labels	NPY file	870 windows
Flatten testing data	NPY file	217 windows
Flatten testing labels	NPY file	217 windows
Classifier model	TensorFlow Lite (float32)	5 KB
Classifier model	TensorFlow Lite (int8 quantized)	3 KB
Classifier model	TensorBoard logs	621 KB
Classifier model	TensorFlow SavedModel	12 KB
Classifier model	Keras h5 model	5 KB
Classifier model	Model evaluation metrics (JSON file)	-

Collaborators (1/4)
haseebay OWNER

Summary
DEVICES CONNECTED: 0
DATA COLLECTED: 18m 11s

Project info
Project ID: 1093590
Labeling method: One label per data item

Figure 6: Edge Impulse block output page: 870 training windows, 217 test windows and the exported classifier models (TensorFlow Lite float32 and int8).

3.13 Importance of Dataset Diversity

Real sensor readings are never perfectly constant, so diversity matters. A model trained on one fixed condition may latch on to details specific to that data and behave differently when conditions shift slightly. The dataset should therefore include reasonable variation in temperature, humidity, illumination, the duration of observed conditions, and ordinary sensor fluctuation. The goal is for the model to learn general patterns for each class rather than memorize individual samples.

3.14 Data Preparation for Embedded Deployment

The dataset and processing configuration also have to suit the ESP32, which has limited RAM, flash storage, processing capability and power. That is a large part of why TinyML was chosen. The objective is not the most complex model possible, but one that classifies usefully and still fits on the target microcontroller.

3.15 Dataset Quality Considerations

Four things received particular attention. First, the sensor channels must stay aligned, so a temperature value always corresponds to the humidity and illumination values from the same moment. Second, labels must accurately describe the condition the data came from, since mislabeled data confuses training. Third, each class should contain enough variation to avoid a narrow representation. Finally, test data should represent conditions the model did not simply memorize during training.

3.16 Data-to-Model Pipeline

The full preparation and learning pipeline is:

$$\begin{aligned} \text{Environment} \rightarrow \text{Sensors} \rightarrow \text{Raw Telemetry} \rightarrow \text{Data Windows} \\ \rightarrow \text{Preprocessing} \rightarrow \text{Features} \rightarrow \text{Classifier} \end{aligned} \quad (7)$$

Once the model is trained and evaluated, the same fundamental pipeline is built into the embedded deployment. This is important because the ESP32 has to process incoming data the same way it was processed during development, so that the model receives input in the format it was trained on.

3.17 Summary

Dataset preparation is the bridge between the physical refrigerator and the model. Temperature, humidity and illumination are collected through the ESP32, organized into labeled observations and turned into input features through Edge Impulse. The resulting model is then deployed to the ESP32, which processes the same kind of data locally in real time. The next section covers the model itself.

4 TinyML Model Development and Deployment

4.1 Overview

With the dataset ready, the next job was to build a model small enough for the ESP32. Unlike models that run on desktops or cloud servers, a TinyML model has to work within the memory and processing limits of a microcontroller, so the design balances classification performance against resource use. Edge Impulse supplied an integrated workflow for preprocessing, feature extraction, training, evaluation and deployment:

$$\text{Dataset} \rightarrow \text{Preprocessing} \rightarrow \text{Feature Extraction} \rightarrow \text{Training} \rightarrow \text{Evaluation} \rightarrow \text{Deployment} \rightarrow \text{ESP32 Inference}$$

4.2 Input Features

The model takes three sensor channels as input: temperature, relative humidity and illumination.

$$X = [T, H, L] \quad (8)$$

With a time-window approach, several consecutive observations form the input:

$$W = \{X_1, X_2, \dots, X_n\} \quad (9)$$

where n is the number of samples in the window.

4.3 Edge Impulse Configuration

The Edge Impulse workflow has three blocks. The *input block* defines the sensor channels, sampling frequency and window size. The *processing block* does the feature extraction and converts raw signals into a form suitable for learning. The *learning block* trains the classifier on the extracted features. The actual parameters go in Table 4.

Table 4: Edge Impulse configuration

Parameter	Value
Input channels	Temperature, Humidity, Light
Sampling frequency	1 Hz
Window size	2 s (two samples per axis)
Window increase	1 s
Processing block	Flatten (21 features)
Learning block	Neural network classifier
Training cycles	59
Batch size	32
Learning rate	0.0005
Training processor	CPU

4.4 Feature Extraction

Raw sensor values are hard for a model to interpret directly, so feature extraction produces a more informative representation:

$$F = g(W) \quad (10)$$

Here W is the sensor window, g the feature extraction operation and F the feature vector. It trims away unnecessary information while keeping useful patterns, which is valuable on a small device. Depending on the processing block, the features may include statistical properties, signal characteristics, and time-domain or frequency-domain features.

4.5 Model Architecture

A lightweight neural network classifier was used. The aims were reliable classification, a fit within the ESP32's memory, efficient execution and acceptable inference speed. The model learns a mapping from features to a class:

$$y = f(F) \quad (11)$$

where F are the extracted features, f is the trained model and y is the predicted class. For this binary problem,

$$y \in \{\text{NORMAL}, \text{ANOMALY}\} \quad (12)$$

The network has a 21-feature input layer, two fully connected hidden layers with 20 and 10 neurons, and an output layer with two classes (Figure 7).

4.6 Training Process

Training is an iterative optimization. The data is presented to the algorithm many times and the model gradually adjusts its parameters. Each pass loads the training data, extracts features, runs forward propagation, calculates the prediction error and updates the parameters. This repeats for many epochs, until the model reaches satisfactory performance or the configured number of training cycles is complete.

Here the model was trained for 59 cycles at a learning rate of 0.0005 on the CPU, with the learned optimizer switched off (Figure 7). The project's training limits (a 60-minute job limit and 8192 MB of training memory, see Appendix D) are far more than a model this small needs.

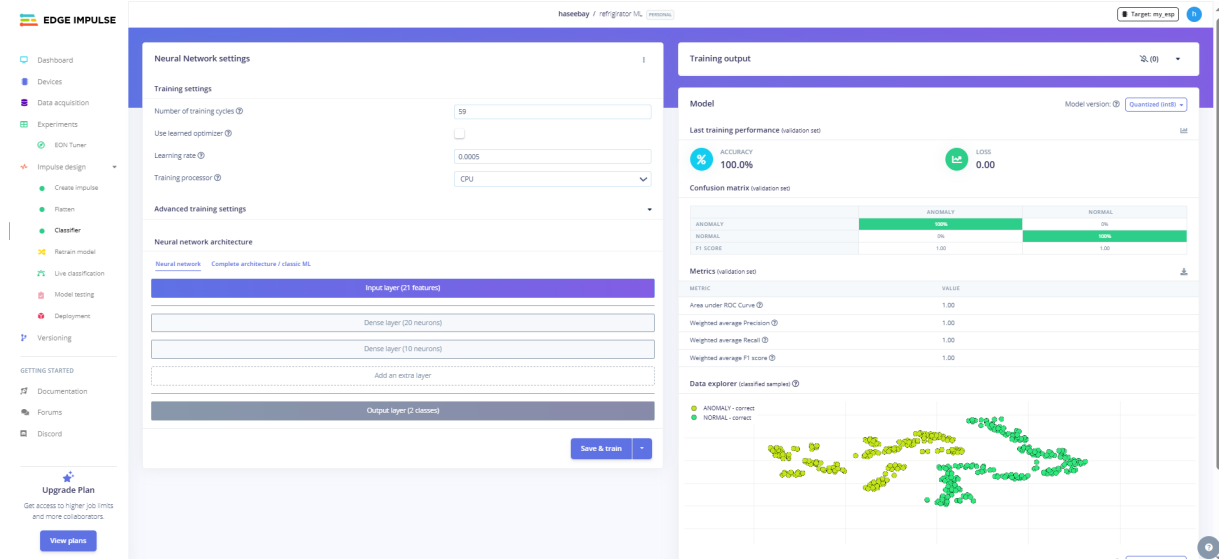


Figure 7: Classifier page in Edge Impulse: training settings, the 21–20–10–2 network and the validation results.

4.7 Training Metrics

Several metrics describe how well the model performs.

4.7.1 Accuracy

Accuracy is the percentage of samples classified correctly:

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \times 100 \quad (13)$$

On the validation set Edge Impulse reports an accuracy of 100 % with a loss of 0.00, and the accuracy on the separate test set is also 100 % (Section 7.5).

4.7.2 Precision

Precision shows how reliable the positive predictions are:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (14)$$

4.7.3 Recall

Recall shows how well the model finds the cases it should find:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (15)$$

4.7.4 F1-Score

The F1-score balances precision and recall:

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (16)$$

4.8 Confusion Matrix

The confusion matrix gives a detailed picture of classification performance. On the validation set every NORMAL window was classified as NORMAL and every ANOMALY window as ANOMALY (100 % in both diagonal cells and 0 % elsewhere), and the F1-score is 1.00 for both classes.

Table 5: Confusion matrix

	Predicted Normal	Predicted Anomaly
Actual Normal	TN	FP
Actual Anomaly	FN	TP

4.9 Classification Output

The model outputs a probability for each class, for example:

```
NORMAL: 0.84
ANOMALY: 0.16
```

The class with the highest probability becomes the final prediction:

$$\text{Prediction} = \arg \max(P_{normal}, P_{anomaly}) \quad (17)$$

4.10 Model Optimization for ESP32

Embedded models must respect hardware limits, so the optimization goals were a small model size, low RAM usage, fast inference and efficient energy use. Edge Impulse’s deployment page estimates the resources for the selected target, for both the quantized (int8) and the unoptimized (float32) model, and these figures are in Table 6. The EON compiler was selected as the inference engine, which Edge Impulse describes as giving the same accuracy with 54 % less RAM and 63 % less ROM.

Table 6: Model resource usage

Metric	Quantized (int8)	Unoptimized (float32)
Flash usage	14.7 KB	13.2 KB
RAM usage	1.4 KB	1.4 KB
Model size (TensorFlow Lite)	3 KB	5 KB
Inference time	1 ms	3 ms

4.11 Model Deployment

After training and evaluation, the model was exported from Edge Impulse in an ESP32-compatible format. Deployment involved exporting the Edge Impulse library, integrating the generated files into the ESP32 firmware, including the required inference functions, preparing the sensor input arrays and running real-time inference:

$$\text{Sensor Data} \rightarrow \text{Feature Processing} \rightarrow \text{Model} \rightarrow \text{Prediction} \quad (18)$$

The model was exported as an Arduino library (build v4, 21 August 2026) with the EON compiler and the quantized int8 model selected (Figure 8). The generated library provides `run_classifier()`, which the firmware calls on the six-value feature signal.

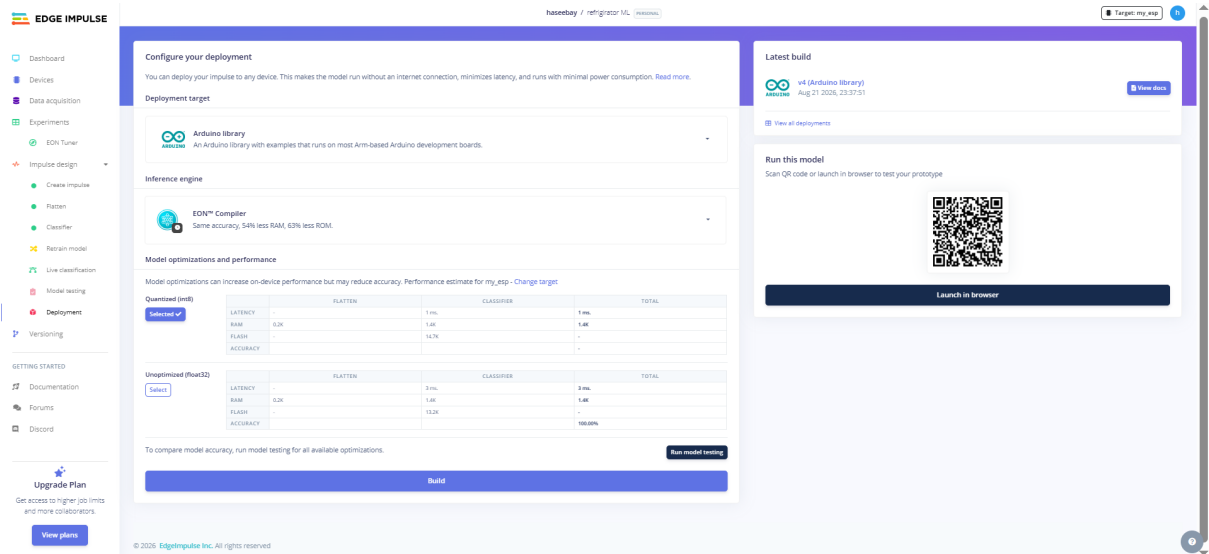


Figure 8: Edge Impulse deployment page: Arduino library, EON compiler and the estimated latency, RAM and flash for the int8 and float32 models.

4.12 Embedded Inference Process

While running, the ESP32 keeps collecting telemetry and repeats one inference cycle: read the sensors, build the input window, apply preprocessing, execute the model, obtain probabilities, select the classification, generate the JSON output and update the dashboard. All of the decision-making happens on the device, with no cloud-based inference.

4.13 Advantages of TinyML Deployment

Running the model on the ESP32 gives local intelligence, less reliance on cloud services, lower communication overhead, faster response, efficient use of resources, and a practical demonstration of Edge AI.

4.14 Summary

This stage turned the collected sensor data into an embedded intelligence system. Using Edge Impulse, a lightweight model was trained, evaluated, optimized and deployed to the ESP32, which can now process temperature, humidity and illumination signals locally and recognize patterns linked to normal and abnormal refrigerator conditions. The next section describes the firmware, JSON telemetry, Wi-Fi communication and dashboard.

5 Embedded Implementation and Real-Time Monitoring

5.1 Overview

Once the model was trained and deployed, it had to be integrated into the ESP32 firmware. The firmware coordinates the entire real-time operation: it reads the sensors, prepares the model input, runs inference, builds a JSON description of the current refrigerator state and sends it over Wi-Fi to the dashboard. The runtime process is:

Initialize → Read Sensors → Prepare Input → Run Inference → Generate JSON → Transmit
Data → Update Dashboard → Repeat

5.2 ESP32 Firmware Structure

Instead of one large block of code, the firmware is split into logical components: hardware initialization, sensor acquisition, data preparation, TinyML inference, classification handling, JSON generation, Wi-Fi communication and dashboard communication. This makes it easier to understand and lets individual parts change without disturbing the rest.

5.3 System Initialization

At startup the firmware initializes serial communication, the DHT22 interface, the LDR input, the Wi-Fi connection, the TinyML inference resources and the services needed by the dashboard. Everything must be ready before continuous monitoring begins.

```
Start
|
+-- Initialize Serial
|
+-- Initialize DHT22
|
+-- Initialize LDR
|
+-- Connect to Wi-Fi
|
+-- Initialize ML Model
```

```
|
+-- Start Monitoring
```

5.4 Sensor Acquisition

After initialization, the ESP32 reads the sensors periodically. The DHT22 provides temperature and relative humidity, and the LDR provides the illumination measurement. The values are held in variables that the inference and telemetry components then use:

```
temperature = readTemperature();
humidity = readHumidity();
light = readLight();
```

The function names here are illustrative; they depend on the actual firmware.

5.5 Sensor Validation

Sensor communication can occasionally return invalid or unavailable measurements, so readings are checked before later stages use them. If a reading is invalid, the firmware can discard it or wait for the next valid one rather than hand bad data to the model. This simple check keeps corrupted values out of the inference pipeline.

5.6 Data Preparation for Inference

Raw sensor values usually cannot go straight into the model. The input must follow the same structure and preprocessing assumptions used in development, so the firmware prepares the data accordingly:

$$\text{Raw Sensor Data} \rightarrow \text{Input Formatting} \rightarrow \text{Preprocessing} \rightarrow \text{Model Input} \quad (19)$$

If the device feeds in data that differs significantly from the training data, the predictions may not mean much.

5.7 TinyML Inference on ESP32

When the input is ready, the ESP32 runs the model, which returns class probabilities. For two classes,

$$P = [P_{normal}, P_{anomaly}] \quad (20)$$

and the predicted class is the one with the highest probability:

$$C = \arg \max(P_{normal}, P_{anomaly}) \quad (21)$$

The firmware uses this result to represent the current refrigerator condition.

5.8 Classification Handling

The application converts the model output into a readable state, either **NORMAL** or **ANOMALY**. An **ANOMALY** means the observed sensor pattern differs from the patterns in the normal class. It should not be read as a specific mechanical fault unless the model has actually been trained and validated to identify that fault.

5.9 Real-Time Telemetry

Besides the model output, the system keeps the latest sensor telemetry: temperature T , humidity H , light L and status C . This lets the dashboard show both the underlying measurements and the model's interpretation of them.

5.10 JSON Telemetry Format

The ESP32 packages the telemetry into a JSON object before sending it. A simplified example:

```
{
  "temp": 3.4,
  "light": 4048,
  "status": "NORMAL",
  "confidence": 0.9810
}
```

These values only illustrate the structure. In the current firmware the payload has four fields: **temp** (one decimal), **light** (the raw ADC value), **status** and **confidence** (four decimals). The firmware also sends an **Access-Control-Allow-Origin** header so that a standalone dashboard page can read the response from the ESP32's IP address (Figure 9). Humidity is measured but is not part of this payload. JSON is a good fit because it is human-readable, simple to integrate with web applications, clearly separates the sensor fields, easy for JavaScript dashboards to parse, and easy to extend with fields such as timestamps, model confidence, device status or connectivity state.

5.11 Wi-Fi Communication

The ESP32 uses its integrated Wi-Fi to talk to the monitoring application. At startup it tries to join the configured network, and once connected it can send the generated telemetry:

$$\text{ESP32} \rightarrow \text{Wi-Fi Network} \rightarrow \text{Monitoring Interface} \quad (22)$$

Wi-Fi is used for telemetry and visualization only, and inference stays local. This split is a core part of the Edge AI design:

$$\text{Inference: Local} \quad \text{Monitoring: Network Connected}$$

So losing the network does not stop the embedded model from running inference.

In the Wokwi simulation the ESP32 joins the Wokwi-GUEST network, prints **WiFi connected!** on the serial monitor and reports the address **http://10.10.0.2**, from which it serves the JSON (Figure 9).

5.12 Dashboard Architecture

The web dashboard is the visualization layer and shows information produced by the ESP32. Data moves from the ESP32 as JSON telemetry to the web interface. The implemented dashboard is a standalone web page with a “Connect Hardware (Serial)” button, a status banner for the current classification, cards for the model's classification confidence, the DSP feature extraction time and the neural network inference time, and a live log of the ESP32 data stream and inference results.

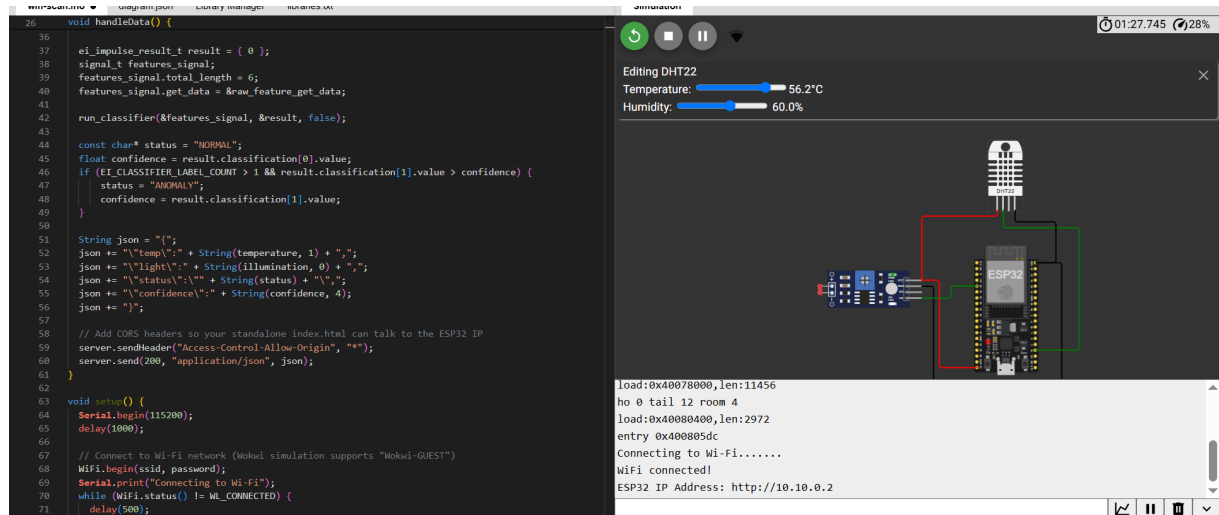


Figure 9: Inference firmware in Wokwi: classification handling and JSON generation (left), Wi-Fi connection and assigned IP address (serial monitor). The DHT22 slider is set to 56.2 °C.

5.13 Dashboard Data Representation

The dashboard presents the telemetry in a friendlier form than raw JSON. A large banner shows the current state and turns into the red message “ALERT: ANOMALY DETECTED” when the model reports an anomaly. Below it, the cards show the confidence of the classification and the time taken by feature extraction and by the neural network, and every inference is appended to a scrolling log with a timestamp, the status and the confidence (Figure 10).

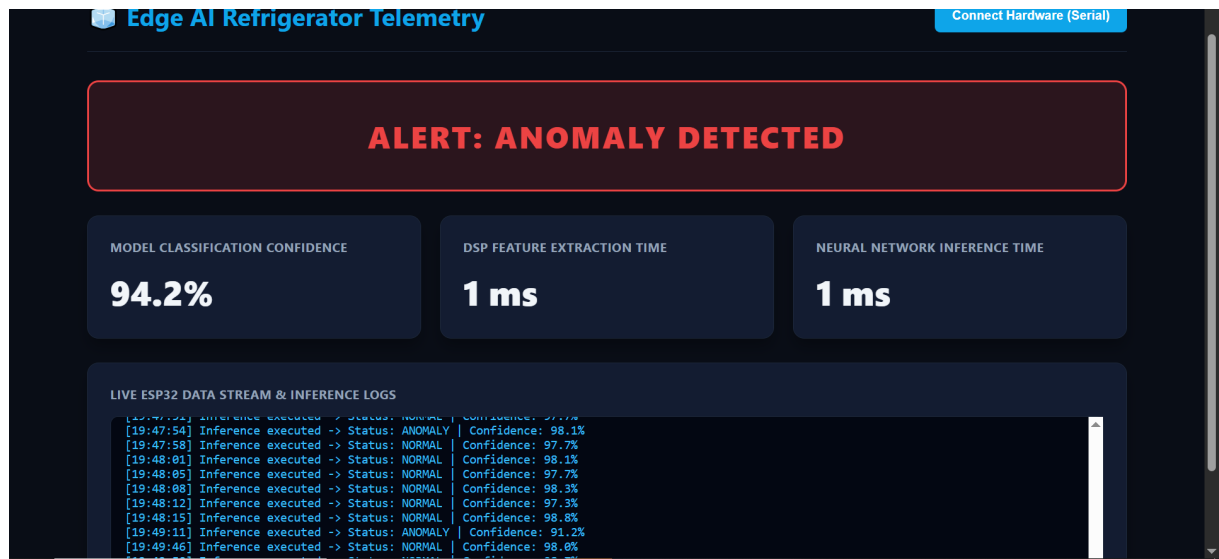


Figure 10: Web dashboard during a test run: anomaly alert at 94.2 % confidence, 1 ms for feature extraction and for inference, and the live inference log.

5.14 Real-Time Monitoring Loop

The system runs continuously rather than once. The loop looks like this:

```
while (system_running)
{
```

```

    readSensors();
    prepareModelInput();
    runInference();
    generateJSON();
    transmitTelemetry();
    updateDashboard();
    waitForNextCycle();
}

```

This keeps the system watching the refrigerator environment and updating the displayed state.

5.15 Monitoring Sequence

One full monitoring cycle goes like this:

1. The ESP32 requests a new measurement from the DHT22.
2. Temperature and humidity are obtained.
3. The LDR measurement is read through the ESP32 input.
4. The sensor values are added to the inference input structure.
5. The TinyML model processes the input.
6. The model produces classification probabilities.
7. The firmware determines the current classification.
8. The sensor readings and the classification are combined.
9. The information is encoded as JSON.
10. The JSON is sent over Wi-Fi.
11. The dashboard displays the latest information.
12. The cycle repeats.

5.16 Local Inference and Remote Visualization

Inference and visualization are deliberately separate. The decision is made locally,

$$\text{Inference}_{local} = f(\text{Sensor Data}) \quad (23)$$

while the monitoring information goes over the network,

$$\text{Telemetry}_{remote} = g(\text{Sensor Data}, \text{Inference}) \quad (24)$$

This is a practical Edge AI pattern: the device does the computation locally and IoT connectivity gives remote visibility.

5.17 Error Handling and Connectivity

The firmware should be ready for temporary Wi-Fi disconnection, invalid sensor readings, delayed sensor responses, incomplete telemetry transmission and initialization failures. A communication failure should not be treated as a machine learning failure. If Wi-Fi drops for a moment, the ESP32 can keep collecting data and running inference while it tries to reconnect, which makes the whole system more robust.

5.18 Wokwi Simulation

Wokwi was part of the development and testing workflow. It provides an environment for exercising firmware logic, sensor interaction and data flow before or alongside physical hardware testing. In simulation it can be used to verify ESP32 firmware execution, sensor reading logic, GPIO configuration, data processing, communication logic and JSON generation. Firmware changes can be tried quickly without touching a physical circuit, which is a big help during iterative development. Figures 2, 3 and 9 show the three stages of the simulation: the potentiometer prototype, the data-collection circuit and the final inference firmware with Wi-Fi.

5.19 Complete Runtime Architecture

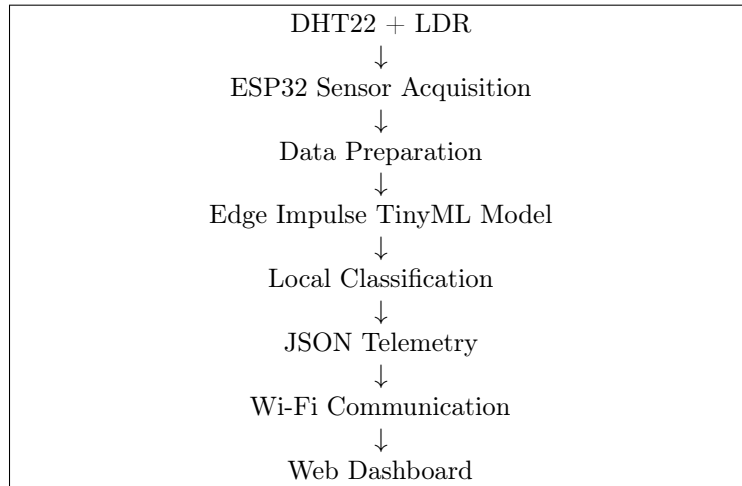


Figure 11: Runtime data flow of the deployed Edge AI system

5.20 System Operation Example

In normal operation the sensors watch the refrigerator environment while the ESP32 collects readings and passes them through the embedded machine learning pipeline. If the pattern looks like the normal training data, the model reports a normal classification. If the combination of measurements departs significantly from the learned normal pattern, it may report an anomaly.

For example, higher illumination together with a change in temperature differs from the normal closed-refrigerator condition. The model uses the combined information and does not rely on a single hand-defined light or temperature threshold. The result goes into the telemetry sent to the dashboard.

5.21 Implementation Summary

The implementation ties every major component into one working pipeline. The ESP32 acts as sensor controller, edge computing device, TinyML inference engine, Wi-Fi communication device and telemetry generator. The dashboard is the visualization layer, so users can see the system state without touching the firmware. Together they show how sensing, embedded processing, machine learning and IoT communication fit into one practical Edge AI application.

6 Testing, Results, and Evaluation

6.1 Overview

After integrating the hardware, deploying the TinyML model, setting up Wi-Fi and building the dashboard, the complete system was tested to confirm the pieces work together. Testing was staged: individual components first, then the machine learning model, then the whole system end to end. The areas covered were sensor functionality, ESP32 firmware operation, data acquisition, TinyML model performance, embedded inference, Wi-Fi communication, JSON telemetry, the web dashboard and end-to-end operation. Working this way made it possible to find problems at the component level before looking at the whole.

6.2 Testing Methodology

The approach was progressive. The sensors were tested first to confirm the ESP32 could obtain valid measurements. Then the data processing and machine learning pipeline was tested. The trained model was evaluated on data that was not used for training, and finally the deployed model and communication components were tested together.

Component Testing → Model Testing → Embedded Testing → Communication Testing →
System Testing

6.3 Sensor Testing

The DHT22 was tested by reading temperature and humidity through the ESP32 and watching the values on the serial interface to confirm the sensor was communicating. The LDR was tested by observing its output under different lighting conditions.

Table 7: Sensor functional test cases

Test	Expected behavior	Result
DHT22 temperature	Valid temperature reading returned	Pass
DHT22 humidity	Valid humidity reading returned	Pass
LDR illumination	Reading changes with illumination	Pass
Repeated readings	Values update continuously	Pass

In Wokwi the DHT22 sliders and the LDR module were used to change the inputs, and the values appeared on the serial monitor, for example 3.40 °C, 62.00 % and an ADC reading of 4048 (Figure 3).

6.4 ESP32 Firmware Testing

The firmware was checked for correct initialization, sensor reading, data processing, model execution, classification output, JSON generation and Wi-Fi communication. Serial output was valuable here, since it let the internal state be observed while the firmware ran. The serial output follows this structure:

```
Temperature: <value>
Humidity: <value>
```

```

Light: <value>
Classification:
  NORMAL: <probability>
  ANOMALY: <probability>
Status: <classification>

```

The serial monitor in Figures 3 and 9 shows sensor rows and the Wi-Fi start-up.

6.5 Machine Learning Model Evaluation

The model was evaluated with Edge Impulse’s built-in evaluation tools, to check whether it can distinguish the classes in the dataset. The metrics considered were accuracy, precision, recall and F1-score, and the final values are in Table 8. They come from the Edge Impulse model testing page, using the float32 model on the held-out test recordings (Figure 12).

Table 8: TinyML model evaluation results

Metric	Measured value
Accuracy	100 %
Precision (weighted average)	1.00
Recall (weighted average)	1.00
F1-score (weighted average)	1.00
Area under ROC curve	1.00

A score of 100 % is encouraging, but it should be read with care. The data comes from a simulation in which the normal and anomaly recordings are quite different, and the test set is only two short recordings (217 windows). The result shows that the pipeline works from end to end. It is not a promise of the same accuracy on a physical refrigerator.

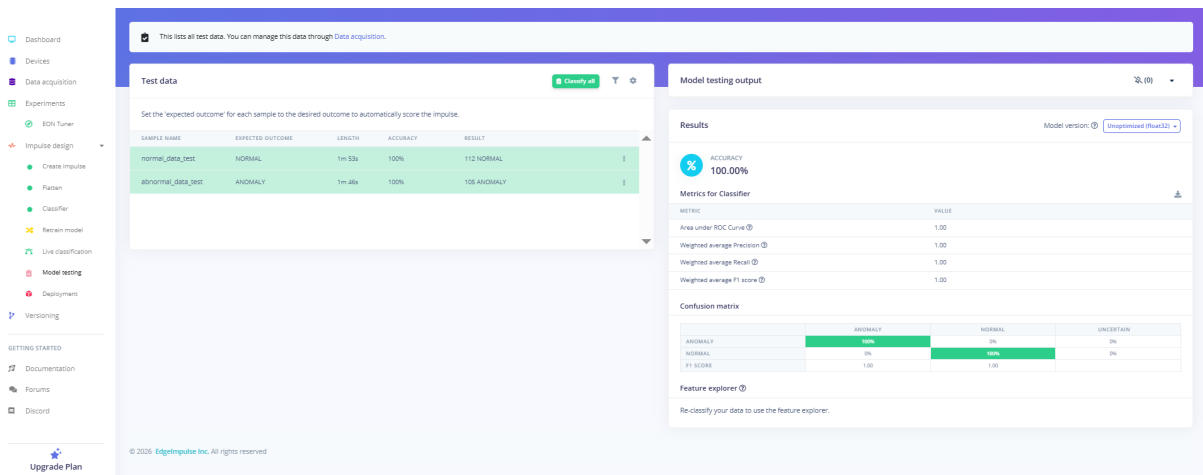


Figure 12: Edge Impulse model testing: both test recordings were classified correctly (112 NORMAL and 105 ANOMALY windows).

6.6 Confusion Matrix Evaluation

For the binary problem, the confusion matrix has four outcomes: true positive (TP), true negative (TN), false positive (FP) and false negative (FN). It is useful because accuracy alone does not reveal which kind of mistake the model makes. A model can score high overall and still raise a lot of false anomaly detections. On the test set, all 112 windows of `normal_data_test` were classified NORMAL and all 105 windows of `abnormal_data_test` were classified ANOMALY. So $TN = 112$, $TP = 105$ and $FP = FN = 0$, and 0 % of the windows were left uncertain.

Table 9: Classification confusion matrix

	Predicted Normal	Predicted Anomaly
Actual Normal	TN	FP
Actual Anomaly	FN	TP

6.7 Model Confidence

The deployed classifier gives a probability for each class, for instance NORMAL: 0.xx and ANOMALY: 0.xx, and the values change with the sensor input at any moment. Confidence is the output of the trained classifier and not a measure of physical certainty. A high probability does not mean the exact physical cause has been identified.

6.8 Normal Condition Testing

In normal-condition testing, the system ran under conditions represented by the normal class. The goal was to check that the sensors produced valid readings, the ESP32 processed the data, the model ran correctly, a classification was produced, telemetry was transmitted and the dashboard displayed the latest information.

Table 10: Normal condition test

Parameter	Observation
Temperature	3.4 °C (DHT22 slider setting in the normal-state run)
Humidity	62.0 % (slider setting)
Illumination	4048 (raw ADC value on the serial monitor)
Model output	NORMAL, confidence 97.3–98.8 % over consecutive inferences
Classification	NORMAL
Dashboard status	Log entries read “Status: NORMAL” (Figure 10)

6.9 Abnormal Condition Testing

Abnormal-condition testing checked whether the system could recognize patterns that differ from normal. The point was not to see whether one sensor crossed a threshold, but how the model responds to the combination of signals. A possible abnormal pattern includes a noticeable change in illumination, a change in temperature over time, associated variation in humidity, or

a combination of changes across channels. The classification was observed through the ESP32 output and the dashboard.

Table 11: Abnormal condition test

Parameter	Observation
Temperature	56.2 °C (slider setting, Figure 9)
Humidity	60.0 % (slider setting)
Illumination	300
Model output	ANOMALY, confidence 94.2 %
Classification	ANOMALY
Dashboard status	Red banner “ALERT: ANOMALY DETECTED” (Figure 10)

6.10 Door-Open Pattern Observation

A door-open event is one of the practical scenarios considered. Opening the door produces a visible change in the light signal, and depending on how long it stays open and on the surroundings, temperature and humidity may change too:

$$\text{Door Event} \rightarrow \Delta\text{Light} + \Delta\text{Temperature} + \Delta\text{Humidity} \quad (25)$$

The system does not measure the mechanical position of the door. It observes the sensor pattern associated with the conditions in the dataset, so the model should only be interpreted in terms of the classes and examples it was trained on.

6.11 Embedded Model Testing

After evaluation in Edge Impulse, the model was deployed to the ESP32 and tested again, because a model that works in development also has to work on the target hardware. The checks covered initialization, input formatting, inference execution, valid classification output and stable repeated inference.

Table 12: Embedded TinyML test

Test item	Expected result	Status
Model initialization	Model loads successfully	Pass
Input generation	Correct model input generated	Pass
Inference	Model produces output	Pass
Classification	Valid class returned	Pass
Continuous operation	Repeated inference works	Pass

6.12 Wi-Fi Communication Testing

Wi-Fi testing verified that the ESP32 could connect to the configured network. It covered network initialization, connection establishment, telemetry transmission, repeated updates and behavior

during a temporary loss of connection:

$$\text{ESP32} \rightarrow \text{Wi-Fi} \rightarrow \text{Telemetry Endpoint} \rightarrow \text{Dashboard} \quad (26)$$

In the simulation the connection succeeded: the serial monitor printed `WiFi connected!` and the ESP32 reported the address `http://10.10.0.2`, from which the JSON is served (Figure 9).

6.13 JSON Telemetry Testing

The JSON was checked to make sure it contained the expected fields with valid values: correct field names and data types, valid JSON structure, updated sensor values and an updated classification status. The same format can later carry timestamps, device identifiers or model confidence.

6.14 Web Dashboard Testing

The dashboard was tested to confirm that information from the ESP32 is displayed correctly.

Table 13: Dashboard test cases

Test case	Expected result	Status
Temperature update	Latest temperature displayed	Pass
Humidity update	Latest humidity displayed	Pass
Light update	Latest illumination value displayed	Pass
Classification update	Current model status displayed	Pass
Repeated updates	Dashboard reflects new telemetry	Pass

In the test run in Figure 10, the log shows a new inference roughly every three to four seconds. Most entries are NORMAL with a confidence between 97 % and 99 %, and the occasional ANOMALY entries have a confidence of 91–98 %.

6.15 End-to-End System Testing

The last stage tested the full system, to confirm that information travels through the whole pipeline without manual intervention:

$$\text{Environment} \rightarrow \text{Sensors} \rightarrow \text{ESP32} \rightarrow \text{TinyML} \rightarrow \text{Classification} \rightarrow \text{JSON} \rightarrow \text{Wi-Fi} \rightarrow \text{Dashboard} \quad (27)$$

The test counted as successful when a sensor condition produced an appropriate inference on the device and the matching information appeared on the monitoring interface.

6.16 End-to-End Test Cases

6.17 Model Resource Evaluation

Because the target is a microcontroller, resource use is a key part of evaluation. Table 15 summarizes the figures from the deployment page (Figure 8). With about 1.4 KB of RAM, 14.7 KB of flash and a 1 ms inference time for the quantized model, the model is clearly small enough for the ESP32. These are Edge Impulse’s estimates for the selected target, and the dashboard shows the same 1 ms for feature extraction and for inference in the simulation.

Table 14: End-to-end system test cases

Test scenario	Expected behavior	Result
Normal environment	System reports normal condition	Pass
Changed illumination	Sensor telemetry changes	Pass
Changed environmental pattern	Model evaluates new pattern	Pass
Telemetry generation	Valid JSON produced	Pass
Wi-Fi transmission	Data reaches monitoring layer	Pass
Dashboard update	Latest data displayed	Pass
Continuous operation	System repeats monitoring cycle	Pass

Table 15: ESP32 TinyML resource evaluation

Resource	Measured value
Flash usage	14.7 KB (int8)
RAM usage	1.4 KB
Inference time	1 ms (int8)
Model size	3 KB (int8 TensorFlow Lite)
Sampling configuration	1 Hz, 2 s window, 1 s step

6.18 Observed Results

The prototype brought sensor monitoring, embedded machine learning and IoT visualization together. The main observations were:

- The ESP32 worked well as the central embedded platform.
- Temperature and humidity could be obtained from the DHT22.
- Illumination changes could be observed with the LDR.
- Sensor data could be prepared for machine learning inference.
- The trained model could be integrated into the ESP32 firmware.
- Local classification could be performed on the device.
- Sensor and inference information could be represented as JSON.
- Wi-Fi carried the telemetry to the monitoring layer.
- The web dashboard gave a convenient view of the current system state.

Quantitative results for the machine learning component should be reported with the actual numbers from the final evaluation.

6.19 Performance Summary

6.20 Limitations

The prototype demonstrates the full Edge AI workflow, but it has limits worth stating plainly.

Table 16: Overall system performance

Parameter	Result
Model accuracy	100 % (test set)
Precision	1.00
Recall	1.00
F1-score	1.00
Inference time	1 ms (int8)
Flash usage	14.7 KB
RAM usage	1.4 KB
Wi-Fi communication	Successful (simulation)
Dashboard monitoring	Successful (simulation)

6.20.1 Limited Sensor Set

Temperature, humidity and illumination give useful information but not a full picture of refrigerator health. Future versions could add more sensors, depending on the intended application.

6.20.2 Dataset Dependency

The model depends on its training data. Conditions that differ greatly from the training examples may not be classified correctly, so adding more real-world conditions to the dataset would improve robustness. Here the dependency is especially strong. The dataset has only four recordings (one normal and one anomaly recording for training, and one of each for testing), all generated in a simulator, and the two classes are well separated. The perfect test scores should therefore be taken as confirmation that the pipeline works, not as an expected accuracy on a real refrigerator.

6.20.3 Prototype Environment

This is a prototype. A real refrigerator brings extra factors such as compressor cycles, airflow, sensor placement, condensation, electromagnetic interference and varying ambient temperature. All of these may affect sensor behavior in a larger physical deployment.

6.20.4 Anomaly Interpretation

The anomaly class means a sensor pattern that differs from the learned normal condition. It does not identify the physical cause. An anomaly could be a door-open pattern, unusual temperature behavior or something else with a similar signature. Finer fault identification would need more training data and probably more sensors.

6.21 Discussion

The results show that several technologies can be combined into one embedded Edge AI workflow. The most important point is that inference runs on the ESP32 itself, while Wi-Fi is used mainly

for telemetry and monitoring and not for sending every sample to a remote machine. In short:

$$\text{Data Acquisition} + \text{Local Intelligence} + \text{IoT Connectivity} = \text{Edge AI Monitoring System} \quad (28)$$

It also shows that a small embedded platform can take part in the whole machine learning lifecycle, from physical data acquisition to inference and user-facing visualization.

6.22 Conclusion of Evaluation

Testing verified the operation of the major components and their integration into one monitoring pipeline. The sensors provide the measurements, the ESP32 performs embedded processing and inference, the TinyML model classifies patterns, Wi-Fi carries telemetry and the dashboard presents it. The quantitative model performance and resource measurements should be reported using the final values from Edge Impulse and the ESP32 deployment.

7 Challenges and Solutions

Building the system raised several challenges across sensor data, machine learning, embedded deployment, communication and integration. Working through them was a large part of getting to a practical Edge AI prototype.

7.1 Sensor Data Variability

Sensor readings are never perfectly stable. Temperature, humidity and light can wobble even when the refrigerator is in the same state, because of sensor characteristics, environmental changes or changes in the simulated environment. To cope, the system processes readings as a group and not as single measurements. Time-based collection and feature extraction help the model find patterns instead of reacting to every small fluctuation.

7.2 Dataset Collection and Labeling

A central challenge is building a dataset that represents what the model will meet in operation. Examples of both normal and abnormal conditions are needed. The data was organized by the operating condition it represented, with normal samples for stable operation and abnormal samples for unusual changes. Careful labeling mattered because wrong labels directly harm training, so the dataset was reviewed before model development.

7.3 Difference Between Normal and Abnormal Conditions

A refrigerator does not run at exactly the same temperature or humidity all the time, and a temporary change is not necessarily a fault. Defining abnormal by a fixed threshold can therefore produce unnecessary alerts. The project handles this with multiple sensor inputs and a learned model, which looks at patterns across temperature, humidity and illumination. This also gives a base for more detailed fault conditions in future work.

7.4 Embedded Resource Constraints

The ESP32 is far more limited than a normal computer, so the model had to be designed around memory, processing demand and deployment size. Edge Impulse was used to create a lightweight

pipeline suited to the ESP32, and unnecessary model complexity was avoided so inference could run within the available resources.

7.5 Consistency Between Training and Deployment

The data used in training has to match the data the ESP32 generates in real-time operation. The same data structure and feature-processing approach used in development must be followed during embedded inference, since significant differences reduce reliability. The shared pipeline is:

Sensor Data → Preprocessing → Feature Extraction → Model Inference → Classification

7.6 Real-Time Inference Integration

Fitting inference into an embedded firmware loop was a challenge of its own, since the ESP32 has to handle sensor acquisition, data processing, inference, classification, Wi-Fi communication and telemetry generation. The firmware was organized into logical stages, with sensor acquisition and inference kept separate from communication and dashboard updates. That makes the system easier to debug and gives a clearer runtime structure.

7.7 Wi-Fi Connectivity

Wi-Fi adds a dependency, and a brief connection failure should not bring the whole application down. Communication handling was therefore kept apart from the core sensing and inference logic, so the system can keep acquiring data and processing locally while communication problems are dealt with separately. This also supports the Edge AI principle of keeping important intelligence close to the data.

7.8 Dashboard Synchronization

The dashboard has to show ESP32 data in a readable and timely way, which needs consistent formatting. JSON was chosen as the communication format for its simple structure. A typical message holds temperature, humidity, illumination and the current classification status.

7.9 Simulation Versus Physical Hardware

The first prototype was built and tested in Wokwi, which is a convenient way to test firmware logic, sensor interfaces, communication and architecture ahead of physical hardware. Simulation cannot reproduce every physical behavior of a real refrigerator or sensor, though. Real hardware adds noise, sensor delays, environmental effects, power issues and communication variation. Physical deployment and more real-world data collection therefore remain important next steps.

7.10 Summary of Challenges

8 Future Improvements

The current system is a working base for Edge AI refrigerator monitoring. Several extensions would make it more practical, more reliable and better suited to real deployment.

Table 17: Major development challenges and approaches

Challenge	Approach
Sensor variability	Use time-based sensor data and multivariate processing.
Dataset preparation	Collect and organize samples by operating condition.
Data labeling	Review and assign labels according to the condition represented.
Embedded constraints	Use a lightweight TinyML model suited to the ESP32.
Training/deployment consistency	Keep the preprocessing and feature pipeline the same in both.
Real-time inference	Separate sensing, inference and communication stages in the firmware.
Wi-Fi reliability	Keep communication logic independent of the core inference process.
Dashboard synchronization	Use structured JSON telemetry for consistent data transfer.
Simulation limitations	Plan physical testing and extra real-world data collection.

8.1 Physical Refrigerator Deployment

The first big step is to install the system on a real refrigerator. Real-world testing would give more representative data and expose the model to conditions that are hard to reproduce in simulation. New datasets from different operating conditions could then be used to improve the model.

8.2 Additional Sensors

More sensors would give a fuller view of the appliance. Possible additions are a door reed switch for direct door-state detection, a current sensor for compressor or appliance power use, voltage monitoring for power-related analysis, extra temperature sensors for different zones, and vibration sensing for compressor condition. These would help the system tell different kinds of abnormal behavior apart.

8.3 Multiclass Fault Detection

At present the system mainly separates normal from abnormal. A future version could recognize several categories, such as normal operation, a possible door-open condition, an unusual temperature increase, excessive temperature fluctuation, a possible cooling-system issue, and sensor-related abnormal behavior. That would turn the binary anomaly detector into a more detailed condition-monitoring system.

8.4 Larger and More Diverse Dataset

Model quality depends heavily on training data quality and variety. Future work should collect more samples under different ambient temperatures, refrigerator loads, door opening durations,

cooling cycles, sensor positions and operating environments. A more varied dataset would make the model less tied to a limited prototype setup.

8.5 Historical Data Visualization

The dashboard could store and chart historical data and not only show the latest readings, with time-series graphs for temperature, humidity, illumination, classification status and detected abnormal events. This would let users spot long-term patterns and review earlier abnormal events.

8.6 Alert and Notification System

A later version could raise alerts when it detects an abnormal condition, through a web interface, mobile app, email or another messaging service, for example:

Warning: Unusual refrigerator condition detected. Please inspect the refrigerator.

The alerting should be designed carefully so brief sensor fluctuations do not cause needless notifications.

8.7 Energy Consumption Monitoring

A current or power sensor would extend the project from condition monitoring to energy monitoring. The system could relate operating conditions to power use and flag unusual energy patterns, which is useful for energy optimization and for keeping an eye on equipment condition.

8.8 Secure IoT Communication

In a real deployment, communication security becomes important. Future versions could use encrypted HTTP or MQTT with proper authentication, reducing the risk of unauthorized access to telemetry or control interfaces.

8.9 Over-the-Air Firmware Updates

An OTA update mechanism would let firmware be delivered to the ESP32 without a programming cable, which is especially helpful when several devices are installed in different places.

8.10 Low-Power Operation

The ESP32 has ample capability for the prototype, but power-efficient operation is worth exploring, especially for battery-powered devices. Options include deep-sleep modes, reduced sampling when conditions are stable, event-driven sensing, optimized Wi-Fi communication and adaptive inference frequency.

8.11 Predictive Maintenance

A longer-term goal is predictive maintenance. Instead of only detecting abnormal conditions, the system could learn patterns that signal gradual wear in refrigerator components. With enough

real-world data, long-term temperature, vibration and energy patterns could point to conditions that need inspection, moving the project from simple telemetry and anomaly detection toward intelligent equipment monitoring.

9 Conclusion

The Edge AI Refrigerator Telemetry System shows how embedded systems, IoT communication and machine learning can work together in a single practical application. An ESP32 collects environmental data through a DHT22 and an LDR, processes it locally and passes it to a TinyML model built with Edge Impulse, which identifies patterns associated with normal and abnormal refrigerator conditions.

The standout feature is that inference runs close to the data source. Instead of streaming raw readings to a server, the ESP32 classifies them itself and then sends the resulting telemetry and status. The project covers the entire workflow:

Sensors → ESP32 → Data Processing → TinyML Model → Local Inference → JSON Telemetry
→ Web Dashboard

Wokwi gave a convenient place to develop and test the embedded system, and Edge Impulse provided the machine learning workflow. Along the way the project ran into practical challenges: dataset preparation, sensor variability, embedded resource limits, communication reliability and the gap between simulation and physical deployment.

The current implementation is a prototype, but it is a solid foundation for real-world refrigerator monitoring. Additional sensors, larger datasets, multiclass fault detection, energy monitoring, secure communication, alerts and predictive maintenance could all extend it considerably.

Overall, the project gave hands-on experience in embedded programming, sensor interfacing, TinyML model development, Edge AI deployment, IoT communication and real-time monitoring. Above all, it demonstrates the full process of taking sensor data from an embedded device, processing it with an on-device model and presenting the result through a monitoring interface.

References

- [1] Espressif Systems, *ESP32 Series Datasheet*, Espressif Systems.
- [2] Espressif Systems, *ESP-IDF Programming Guide*, Espressif Systems.
- [3] Edge Impulse, *Edge Impulse Documentation and Embedded Machine Learning Platform*, Edge Impulse.
- [4] Wokwi, *Wokwi Online Electronics Simulator Documentation*, Wokwi.
- [5] Aosong Electronics, *AM2302/DHT22 Digital Temperature and Humidity Sensor Datasheet*, Aosong Electronics.
- [6] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*, O'Reilly Media.
- [7] TensorFlow, *TensorFlow Lite for Microcontrollers Documentation*, Google.

A Hardware Connections

Table 18 documents the final connections for the physical or simulated build. The GPIO numbers are those of the final Wokwi circuit; the first prototype used potentiometers on GPIO 34, 35 and 32.

Table 18: Sensor and ESP32 connections

Component	ESP32 interface	Purpose
DHT22	GPIO 15	Temperature and humidity measurement
LDR	ADC GPIO 34	Illumination measurement
Wi-Fi	Built-in	Wireless telemetry communication
Power	3.3 V / 5 V	Component power supply
GND	GND	Common electrical reference

B Example JSON Telemetry

This is an example of the JSON the embedded system generates. The values are illustrative and are not final experimental measurements.

```
{
  "temp": 3.4,
  "light": 4048,
  "status": "NORMAL",
  "confidence": 0.9810
}
```

The structure can be extended with timestamps, device identifiers, model confidence, battery information or additional sensor measurements.

C Firmware Structure

The firmware follows this logical structure:

```
Initialize ESP32
|
+-- Initialize sensors
|
+-- Initialize serial communication
|
+-- Connect to Wi-Fi
|
+-- Initialize TinyML model
|
v
Read sensor values
|
v
Validate sensor readings
|
```

```

v
Prepare input data
|
v
Run TinyML inference
|
v
Generate classification
|
+-----+
| |
v v
Generate Update
JSON Dashboard
Telemetry
|
v
Repeat

```

D Edge Impulse Model Report

Table 19 collects the main numbers from the Edge Impulse project in one place.

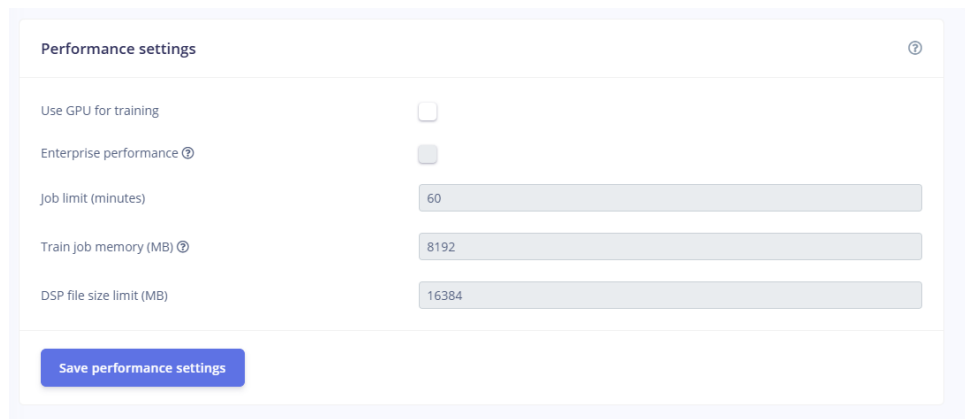
Table 19: Edge Impulse project summary

Item	Value
Data collected	18 min 11 s (80 %/20 % train/test split)
Windows	870 training, 217 test
Classes	NORMAL, ANOMALY
Sampling and window	1 Hz, 2 s window, 1 s step
Processing block	Flatten, 21 features
Learning block	Neural network: 21 inputs, dense 20, dense 10, 2 outputs
Training	59 cycles, learning rate 0.0005, CPU
Validation result	100 % accuracy, loss 0.00
Test result	100 % accuracy, 112 NORMAL and 105 ANOMALY windows correct
Precision, recall, F1	1.00 each (weighted average)
Model size (TFLite)	3 KB int8, 5 KB float32
RAM, flash, latency (int8)	1.4 KB, 14.7 KB, 1 ms

E Final Project Summary

The whole project comes down to one pipeline:

Sense → Collect → Process → Learn → Infer → Communicate → Monitor



The screenshot shows the 'Performance settings' window in the Edge Impulse IDE. It contains five settings: 'Use GPU for training' (unchecked), 'Enterprise performance' (disabled), 'Job limit (minutes)' (60), 'Train job memory (MB)' (8192), and 'DSP file size limit (MB)' (16384). A 'Save performance settings' button is at the bottom.

Setting	Value
Use GPU for training	☐
Enterprise performance	☐
Job limit (minutes)	60
Train job memory (MB)	8192
DSP file size limit (MB)	16384

Figure 13: Edge Impulse performance settings for the project (CPU training, 60-minute job limit, 8192 MB training memory).

ESP32 + DHT22 + LDR + TinyML + Wi-Fi + JSON + Web Dashboard

It is a complete Edge AI workflow in which sensor data is acquired by an embedded device, processed locally, analyzed by a machine learning model and shown through a real-time monitoring interface.